PROJECT PHASE 1 REPORT

AGENTIC SOC: AN AUTONOMOUS LEVEL-1 SECURITY ANALYST USING ENSEMBLE MACHINE LEARNING, OSINT THREAT INTELLIGENCE AND LLM-BASED REASONING

Submitted by

THARUN S V

in partial fulfilment of the requirements for the degree of

MASTER OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

II Year

October 2026

TABLE OF CONTENTS

1. TITLE 2
2. ABSTRACT 3
3. BASE PAPER 4
3.1 Summary 4
3.2 Methodology 4
3.3 Limitations Addressed by This Project 4
3.4 Extension in This Project 4
4. LITERATURE SURVEY 5
4.1 Research Gap 6
5. PROJECT REPORT 7
5.1 Introduction 7
5.2 Problem Statement 7
5.3 Objectives 7
5.4 Existing System vs Proposed System 7
5.5 System Architecture 8
5.6 Modules 8
5.7 System Requirements 9
5.8 Methodology 9
6. IMPLEMENTATION STATUS (50%) 11
6.1 Completed in Phase 1 11
6.2 Results 11
6.3 Work Planned for Phase 2 12
REFERENCES 13

1. TITLE

AGENTIC SOC: AN AUTONOMOUS LEVEL-1 SECURITY ANALYST USING ENSEMBLE MACHINE LEARNING, OSINT THREAT INTELLIGENCE AND LLM-BASED REASONING

Domain: Cyber Security / Machine Learning

Student: THARUN S V, M.E. Computer Science and Engineering, II Year

2. ABSTRACT

Security Operations Centres (SOCs) receive thousands of alerts a day, and Level-1 analysts spend hours triaging each one by hand. Signature-based tools miss new and disguised attacks, while pure machine-learning detectors raise alerts without explaining them. This slows the Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR).

This project builds an Agentic SOC that automates the full Level-1 workflow: detect, explain, decide and contain. Incoming network and authentication logs first pass an OSINT stage that matches IPs, domains, ports and hashes against known threat indicators. Logs then go to an ensemble detector: a Random Forest classifier for known attack types and an Isolation Forest trained only on normal traffic to catch zero-day outliers. The Random Forest is trained on CIC-IDS2017 data mapped to a common log schema and augmented with adversarial perturbations, so attackers who try to blend in are still flagged. SHAP values identify which features drove each decision.

Flagged events go to a Large Language Model (Claude), which returns a containment action (isolate, revoke or honeypot), a plain-language narrative, a YARA rule and a step-by-step justification. The system then generates a firewall playbook script and shows the incident on a live web dashboard. Analysts can undo actions and mark false positives, which feed a retraining pipeline.

On a held-out test set of 21,673 events, the detector reaches 96% accuracy, 0.94 precision and 0.96 recall on the anomaly class. The project shows that combining ML detection, explainability and LLM reasoning can cut Level-1 triage from hours to seconds.

Keywords: intrusion detection, Random Forest, Isolation Forest, SHAP, OSINT, SOAR, large language models, CIC-IDS2017.

3. BASE PAPER

I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP), Funchal, Portugal, 2018, pp. 108–116.

3.1 Summary

The paper introduces the CIC-IDS2017 dataset from the Canadian Institute for Cybersecurity. It captures five days of realistic benign traffic plus attacks: brute force (FTP/SSH), DoS, DDoS, Heartbleed, web attacks, infiltration, botnet and port scan. CICFlowMeter extracts more than 80 flow features per connection.

3.2 Methodology

The authors profile human user behaviour to generate benign background traffic, run the attacks on a test network, label each flow, and evaluate seven ML classifiers. They use Random Forest feature importance to pick the best features for each attack. Random Forest, ID3 and KNN give the best precision and recall.

3.3 Limitations Addressed by This Project

- The paper stops at detection. It has no explanation of individual alerts, no response action and no analyst workflow.
- Models are trained only on known, labelled attacks. They are not tested against unseen (zero-day) or adversarially disguised traffic.
- There is no threat-intelligence input; every flow is judged on its features alone.

3.4 Extension in This Project

We use CIC-IDS2017 as the training source and keep Random Forest as the core classifier. On top of it we add an Isolation Forest for zero-day outliers, adversarial training data, SHAP explanations, an OSINT pre-filter, LLM-generated containment decisions and an automated response layer.

4. LITERATURE SURVEY

Eight papers cover the four building blocks of this project: classical ML detection, anomaly detection, explainability and adversarial robustness.

#	Paper	Technique	Key Finding	Limitation / Gap	Use in Project
1	L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.	Ensemble of decision trees with bagging and random feature selection	Lower variance and better accuracy than single trees; gives feature importance	Supervised only; cannot flag unseen attack types	Core classifier for known attacks
2	F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation Forest," Proc. IEEE ICDM, 2008, pp. 413–422.	Unsupervised anomaly detection by random partitioning	Linear time, low memory, needs no attack labels	Higher false positives alone; no reason for a flag	Zero-day detector trained on normal traffic
3	A. L. Buczak and E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," IEEE Commun. Surveys & Tutorials, vol. 18, no. 2, 2016.	Survey of ML/DM methods for misuse and anomaly detection	Hybrid misuse + anomaly systems perform best	Lack of realistic datasets and real-time evaluation	Justifies hybrid RF + IF design
4	R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," Proc. IEEE S&P, 2010, pp. 305–316.	Critical analysis of ML-based IDS	High false-positive cost; operators cannot act on unexplained alerts	Position paper; no system built	Motivates SHAP + LLM explanations and feedback
5	Y. Mirsky et al., "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection," Proc. NDSS, 2018.	Online unsupervised ensemble of autoencoders	Real-time detection without labels	No explanation or response; hard to audit	Supports online per-event ensemble scoring
6	S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," Proc. NeurIPS, 2017.	SHAP: Shapley-value feature attribution	Consistent per-prediction explanations; fast TreeSHAP	Explains features, not the attack story	Top features per alert passed to the LLM
7	I. J. Goodfellow, J. Shlens and C. Szegedy, "Explaining and Harnessing Adversarial	Adversarial examples and adversarial training	Training on perturbed samples improves robustness	Studied on images, not network traffic	Adversarial augmentation of attack samples

#	Paper	Technique	Key Finding	Limitation / Gap	Use in Project
	Examples," Proc. ICLR, 2015.				
8	M. Ring et al., "A Survey of Network-based Intrusion Detection Data Sets," Computers & Security, vol. 86, pp. 147–167, 2019.	Survey of 34 IDS datasets	Recommends recent realistic datasets like CIC-IDS2017	Datasets are still lab-generated	Supports choice of CIC-IDS2017

Table 4.1: Summary of literature survey

4.1 Research Gap

Existing work treats detection, explanation and response as separate problems. No surveyed system takes a raw log through threat-intelligence lookup, ensemble detection, explanation, a containment decision and an executed playbook in one automated pipeline. This project fills that gap.

5. PROJECT REPORT

5.1 Introduction

Modern attacks are polymorphic, fileless and often automated, so signature matching alone no longer works. SOC teams rely on Level-1 analysts to read each alert, decide whether it is real and take a first containment step. With alert volumes in the thousands per day, this manual triage is the main bottleneck in incident response. The Agentic SOC acts as an autonomous Level-1 analyst that handles that first step in seconds and hands humans a clear, explained decision.

5.2 Problem Statement

To design an automated system that analyses streams of network and authentication logs, detects malicious behaviour (including unseen and disguised attacks), explains why each event was flagged, and triggers a containment action without human intervention, while keeping false positives low and letting analysts override and correct the system.

5.3 Objectives

1. Build an ML detection engine trained on CIC-IDS2017 that balances precision and recall.
2. Detect zero-day and adversarial traffic using an unsupervised model and adversarial training.
3. Cross-check every log against OSINT threat indicators before ML scoring.
4. Explain each alert with SHAP feature attributions and an LLM-written narrative.
5. Automate response: choose isolate / revoke / honeypot and generate a firewall playbook.
6. Show live incidents, severity and actions on a web dashboard.
7. Learn from analyst feedback (false positive / false negative) through retraining.

5.4 Existing System vs Proposed System

Aspect	Existing System (Signature IDS + Manual Triage)	Proposed Agentic SOC
Detection	Signature rules; misses new variants	RF + Isolation Forest ensemble on behavioural features
Zero-day / evasion	Not detected until a rule is written	Isolation Forest outliers + adversarially trained RF

Aspect	Existing System (Signature IDS + Manual Triage)	Proposed Agentic SOC
Threat intelligence	Separate tool, checked by hand	OSINT lookup built into the pipeline, runs first
Explanation	Alert ID only	SHAP top features + LLM narrative and reasoning steps
Response	Manual; minutes to hours	Automatic containment + firewall script; seconds
Learning	Rules updated by hand	Feedback endpoint + retraining pipeline

Table 5.1: Comparison of existing and proposed systems

5.5 System Architecture

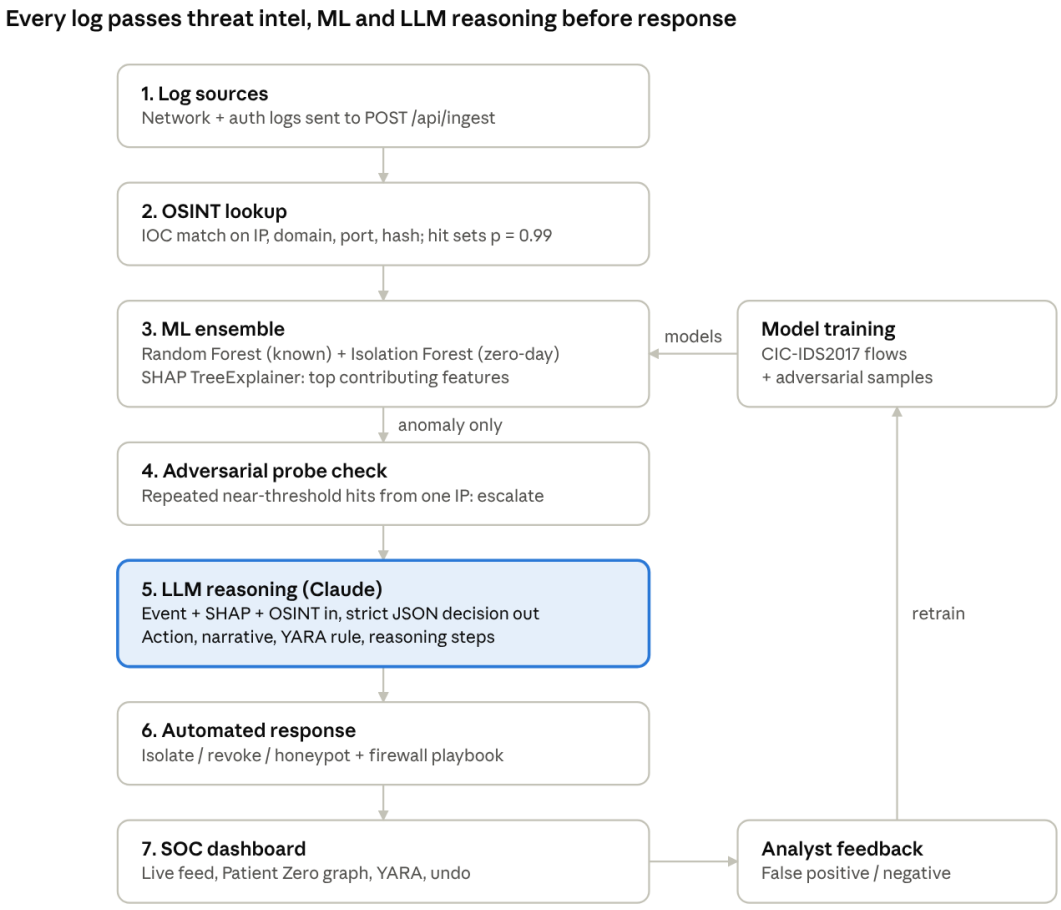


Figure 5.1: Architecture of the Agentic SOC

Each log moves top to bottom in one request; only events flagged as anomalous reach the LLM. Analyst corrections flow back into model training.

5.6 Modules

Module	File(s)	Responsibility
Data preparation	prepare_cic_data.py, log_generator.py	Map CIC-IDS2017 flows to a common log schema; generate synthetic stolen-token and lateral-movement attacks
Model training	train_model.py	Feature encoding, adversarial augmentation, train Random Forest + Isolation Forest, save models
OSINT threat intelligence	osint.py	Look up IPs, domains, ports and file hashes against IOC lists
Detection API	app.py	/api/ingest: OSINT → ML scoring → SHAP → probe check → incident creation
LLM reasoning	claude_reasoning.py	Return action, narrative, YARA rule, ROI estimate; safe fallback if the API fails
Response (SOAR)	app.py	Generate isolate / revoke / honeypot firewall scripts; undo endpoint
Feedback & retraining	app.py, retrain_pipeline.py	Store analyst corrections; retrain models
Dashboard	templates/, static/	Login, Patient Zero graph, live feed, YARA panel, undo button
Simulation	demo.py	Stream logs into the API to demonstrate the pipeline

Table 5.2: System modules

5.7 System Requirements

Software: Python 3.10, Flask, scikit-learn, pandas, NumPy, joblib, SHAP, Anthropic Python SDK; HTML, CSS and JavaScript with vis-network; Windows or Linux.

Hardware: x86-64 machine with 8 GB RAM (16 GB recommended for training on the full CIC-IDS2017 set) and internet access for LLM calls.

Dataset: CIC-IDS2017 (Canadian Institute for Cybersecurity) and generated synthetic logs.

5.8 Methodology

1. **Data preparation.** CIC-IDS2017 CSVs are cleaned and sampled to balance benign and attack flows (up to 20,000 rows per file), then mapped to a common schema: IPs, ports, protocol, user, asset, action, outcome, bytes sent, duration, failed logins, privilege and request frequency.
2. **Adversarial augmentation.** Each of the 35,346 training anomalies is perturbed to mimic attackers blending into normal traffic, expanding the training set to 192,729 samples.
3. **Ensemble training.** A Random Forest learns known attack patterns. An Isolation Forest is trained on 64,182 normal-only samples, so anything far from normal is flagged even without a label.

4. **Real-time ingestion.** Each log first goes through OSINT lookup; a confirmed IOC sets anomaly probability to 0.99. Otherwise the ensemble scores it and SHAP TreeExplainer computes the top contributing features.
5. **Adversarial probe check.** Repeated near-threshold events from one source IP are treated as boundary probing and escalated.
6. **LLM reasoning.** Anomalous events, SHAP features and OSINT hits are sent to Claude with a constrained prompt that must return strict JSON.
7. **Automated response.** The chosen action is recorded, a firewall playbook script is generated and the incident appears on the dashboard.
8. **Feedback loop.** Analysts can undo an action or mark a false positive / negative; corrections feed the retraining pipeline.

6. IMPLEMENTATION STATUS (50%)

The end-to-end prototype runs: a log travels from ingestion to a dashboard incident with a generated playbook. Phase 2 hardens it for real deployment and completes the evaluation.

6.1 Completed in Phase 1

Component	Status	Evidence
CIC-IDS2017 preprocessing + synthetic attack generator	Completed	prepare_cic_data.py, log_generator.py
RF + Isolation Forest training with adversarial augmentation	Completed	train_model.py, models/, metrics.txt
SHAP explanations per alert	Completed	TreeExplainer in app.py
OSINT IOC lookup (IP, domain, port, hash)	Completed (static IOC list)	osint.py, test_osint.py
Flask detection API with adversarial-probe check	Completed	app.py: /api/ingest, /api/state
LLM reasoning with JSON output and fallback	Completed	claude_reasoning.py
Containment actions, playbook generation, undo	Completed (not executed)	/api/action/*, playbooks/
Analyst feedback + retraining script	Completed (basic)	/api/feedback, retrain_pipeline.py
Dashboard and login page	Completed (prototype)	templates/, static/
Simulation runner	Completed	demo.py

Table 6.1: Phase 1 implementation status

6.2 Results

Test set: 21,673 events (12,837 normal, 8,836 anomalous), after training on 192,729 samples including adversarial examples.

Class	Precision	Recall	F1-Score
Normal	0.97	0.96	0.97
Anomaly	0.94	0.96	0.95
Overall accuracy			0.96

Table 6.2: Classification results

	Predicted Normal	Predicted Anomaly
Actual Normal	12,310 (TN)	527 (FP)
Actual Anomaly	339 (FN)	8,497 (TP)

Table 6.3: Confusion matrix

The false-positive rate is 4.1% of normal traffic and 3.8% of attacks are missed.

6.3 Work Planned for Phase 2

- Tamper-proof blockchain audit log of every incident and action.
- Live OSINT feeds (e.g. AbuseIPDB, AlienVault OTX) instead of a static IOC list.
- Execute playbooks against a real firewall in a sandbox virtual machine.
- Persistent database for incidents and feedback (currently in memory).
- Server-side authentication with hashed passwords.
- Migrate the dashboard to the React frontend.
- Full evaluation: per-attack-type results, end-to-end latency benchmark, baseline comparison and adversarial evasion tests.
- Final report, user manual and demonstration video.

REFERENCES

1. I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in Proc. ICISSP, 2018, pp. 108–116.
2. L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
3. F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation Forest," in Proc. IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413–422.
4. A. L. Buczak and E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," IEEE Commun. Surveys & Tutorials, vol. 18, no. 2, pp. 1153–1176, 2016.
5. R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," in Proc. IEEE Symp. Security and Privacy, 2010, pp. 305–316.
6. Y. Mirsky, T. Doitshman, Y. Elovici and A. Shabtai, "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection," in Proc. NDSS, 2018.
7. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Proc. NeurIPS, 2017, pp. 4765–4774.
8. I. J. Goodfellow, J. Shlens and C. Szegedy, "Explaining and Harnessing Adversarial Examples," in Proc. ICLR, 2015.
9. M. Ring, S. Wunderlich, D. Scheuring, D. Landes and A. Hotho, "A Survey of Network-based Intrusion Detection Data Sets," Computers & Security, vol. 86, pp. 147–167, 2019.
10. Canadian Institute for Cybersecurity, "Intrusion Detection Evaluation Dataset (CIC-IDS2017)," https://www.unb.ca/cic/datasets/ids-2017.html